Trabalho Prático 03

João Gabriel Guimarães Alves Vaz

Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2025

1 Introdução

Neste trabalho, enfrentamos um problema de otimização inspirado no xadrez. Dado um tabuleiro com obstáculos, uma rainha e diversos peões distribuídos em posições variadas, o objetivo é determinar o menor número de jogadas necessárias para que a rainha capture todos os peões. A rainha se move conforme as regras tradicionais do xadrez: em qualquer direção ortogonal ou diagonal, por quantas casas quiser, desde que não ultrapasse obstáculos ou peões.

Trata-se de um problema computacionalmente desafiador, pois exige a exploração eficiente de todas as possibilidades de movimentação da rainha ao longo do tabuleiro, com restrições topológicas impostas pelos obstáculos. Além disso, como o número de peões pode chegar a 16, o espaço de busca cresce rapidamente, especialmente se considerarmos todas as possíveis ordens de captura.

Com isso, este trabalho propõe duas soluções distintas para o problema: uma abordagem exata, que garante encontrar a sequência ótima de capturas, e uma abordagem aproximada, que busca uma solução eficiente com boa qualidade. Ambas as soluções são discutidas e comparadas em termos de precisão e desempenho ao longo desta documentação.

Abaixo está um resumo do que será abordado em cada uma das sessões desse documento.

- Na seção 1, há uma introdução ao tema e este sumário;
- Na seção 2, há detalhes da modelagem;
- Na seção 3, há a solução para o poblema;
- Na seção 4, há uma análise de complexidade;
- Na seção 5, há a discussão dos algoritmos;
- Na seção 6, há as referências.

2 Modelagem

Para resolver o problema computacional proposto, foi necessário transformar o cenário do tabuleiro em estruturas manipuláveis por algoritmos. O tabuleiro foi representado como uma matriz $N \times M$, e as células foram mapeadas por coordenadas (i, j). As células intransponíveis, os peões e a posição inicial da rainha foram identificados e tratados com restrições específicas. Em ambas as abordagens, a *distância* foi definida como o número mínimo de jogadas completas necessárias para alcançar determinada posição no tabuleiro.

2.1 Solução Exata

Na abordagem exata, eu modelei o problema como uma variante do *Problema do Caixeiro Viajante (TSP)*. A ideia central foi considerar a rainha como um agente que precisa visitar todos os peões no menor número possível de jogadas, partindo da sua posição inicial. Para isso, construí uma matriz de distâncias entre todos os pares relevantes: a posição da rainha e as posições dos peões.

Cada valor dessa matriz representa a menor quantidade de jogadas que a rainha precisa fazer para ir de um ponto a outro, considerando os obstáculos do tabuleiro. Para obter essas distâncias, utilizei uma simulação dos movimentos da rainha, onde a rainha avança em linha reta por cada direção possível, acumulando um passo apenas ao mudar de direção. Com essa matriz de distâncias em mãos, apliquei o algoritmo de Held-Karp com programação dinâmica e bitmask para encontrar a menor sequência de capturas possível. Esse algoritmo considera todos os subconjuntos de peões visitados e o último peão capturado, atualizando a solução ótima para cada configuração possível.

2.2 Solução Aproximada (Gulosa)

Na solução aproximada, adotei uma estratégia gulosa baseada na captura sucessiva do peão mais próximo. Para isso, modelei o tabuleiro como um *grafo não direcionado*, onde cada vértice representa uma célula acessível pela rainha, e arestas ligam pares de casas que podem ser conectadas em uma única jogada legal, sem atravessar obstáculos.

Esse grafo foi construído adicionando, para cada casa livre do tabuleiro, conexões com todas as demais casas visíveis ao longo das oito direções possíveis de movimentação da rainha. Os obstáculos impedem a criação dessas conexões.

Durante a execução do algoritmo, a cada jogada, eu realizo uma *busca em largura (BFS)* no grafo a partir da posição atual da rainha, encontrando o peão mais próximo — no sentido do menor número de jogadas necessárias. Esse peão é então capturado, a posição da rainha é atualizada, e o processo é repetido até que todos os peões tenham sido removidos do tabuleiro, ou até que não seja mais possível alcançar os restantes.

Essa abordagem não garante a solução ótima, mas oferece uma boa aproximação em tempo polinomial, funcionando como um limite superior para a quantidade mínima de jogadas necessárias.

3 Solução

Nesta seção, eu descrevo o funcionamento geral de cada uma das soluções propostas: a exata, baseada em programação dinâmica com bitmask (algoritmo de Held-Karp), e a aproximada, que adota uma abordagem gulosa. Ambas as estratégias utilizam, inicialmente, uma análise do tabuleiro para simular os movimentos válidos da rainha e estruturar as distâncias entre os elementos relevantes.

3.1 Solução Exata

A solução exata emprega uma adaptação do algoritmo de Held-Karp, que resolve o problema do Caixeiro Viajante (TSP) com complexidade exponencial. Primeiramente, eu construo uma matriz D de distâncias entre os $k+1$ pontos relevantes do tabuleiro: a posição inicial da rainha e os k peões. A entrada $D[i][j]$ representa o número mínimo de jogadas que a rainha precisa fazer para ir do ponto i ao ponto j, sem atravessar obstáculos.

Essas distâncias são calculadas por meio de uma busca, na qual a rainha se movimenta como no xadrez tradicional: em qualquer direção reta até encontrar um obstáculo, um peão ou a borda do tabuleiro. Cada avanço completo em linha reta é contado como uma jogada.

Com essa matriz de distâncias D construída, aplico o algoritmo de Held-Karp com programação dinâmica e bitmask. A ideia é calcular o menor custo para visitar todos os peões a partir da rainha. A estrutura $dp[\text{mask}][u]$ guarda o menor custo para visitar os peões indicados por `mask` terminando no peão u.

Algorithm 1 Algoritmo de Held-Karp com Bitmask

Input: matriz D de distâncias entre rainha e peões
Inicializar $dp[2^k][k] \leftarrow \infty$
for $i \leftarrow 0$ **to** $k-1$ **do**
  $dp[1 \ll i][i] \leftarrow D[0][i+1]$
end for
for mask = 1 to $2^k - 1$ **do**
  for u = 0 to $k-1$ **do**
    if mask does not include u **then**
      continue
    end if
    for v = 0 to $k-1$ **do**
      if mask includes v **then**
        continue
      end if
      $next \leftarrow mask \mid (1 \ll v)$
      $dp[next][v] \leftarrow \min(dp[next][v], dp[mask][u] + D[u+1][v+1])$
    end for
  end for
end for
Output: $\min(dp[2^k - 1][i])$ para todo i

3.2 Solução Aproximada (Gulosa)

Na solução aproximada, eu utilizo um algoritmo guloso baseado em buscas sucessivas pelo peão mais próximo. O tabuleiro é transformado em um grafo não direcionado, onde cada vértice representa uma célula livre, e duas células são conectadas por uma aresta se a rainha pode alcançá-las diretamente (em uma única jogada, sem cruzar obstáculos).

Algorithm 2 Construção do grafo a partir do tabuleiro

for cada celula (i, j) no tabuleiro **do**
  if celula (i, j) eh obstaculo **then**
    continue
  end if
  for cada direcao (dx, dy) **do**
    Avance $(ni, nj) \leftarrow (i + dx, j + dy)$ ate sair do tabuleiro ou encontrar obstaculo
    Crie arestas entre (i, j) e cada celula acessada (ni, nj)
  end for
end for

Durante a execução do algoritmo guloso, a cada passo executo uma BFS no grafo a partir da posição da rainha, encontro o peão mais próximo em número de jogadas, atualizo a posição da rainha e removo o peão. O processo é repetido até que todos os peões sejam capturados ou até que não haja caminho.

Algorithm 3 Laço principal do algoritmo guloso

$moves \leftarrow 0$
while existem peões não capturados **do**
  Executar **BFS** a partir da posição atual da rainha
  Localizar o peão mais próximo na ordem de camadas da BFS
  if nenhum peão encontrado **then**
    Retornar -1
  end if
  Capturar o peão encontrado
  Atualizar posição da rainha
  $moves \leftarrow moves+$ camadas percorridas
end while
Retornar $moves$

4 Análise de Complexidade

Nesta seção, eu analiso o desempenho assintótico das soluções propostas, considerando tanto a complexidade temporal quanto a espacial. Para isso, utilizo as notações usuais $\mathcal{O}$ para expressar limites superiores.

4.1 Tempo

4.1.1 Solução Exata

A solução exata consiste em duas fases principais: o pré-processamento das distâncias entre os $k+1$ pontos relevantes (rainha e peões), e a execução do algoritmo de Held-Karp sobre essa matriz de distâncias.

- Para cada um dos $k+1$ pontos, eu executo uma busca para simular os movimentos da rainha em linha reta em todas as direções. O tempo de cada busca é $\mathcal{O}(NM)$, o que resulta em um total de $\mathcal{O}((k+1) \cdot NM)$ para essa etapa.
- O algoritmo de Held-Karp utiliza uma tabela de programação dinâmica de tamanho $2^k \cdot k$, e para cada entrada tenta transições para k estados futuros, totalizando complexidade temporal $\mathcal{O}(2^k \cdot k^2)$.

Assim, a complexidade total de tempo da solução exata é:

$$\mathcal{O}((k+1) \cdot NM + 2^k \cdot k^2)$$

4.1.2 Solução Aproximada

A solução aproximada executa, a cada iteração, uma BFS a partir da posição atual da rainha até encontrar o peão mais próximo. Para isso:

- O grafo é construído com base nas conexões válidas entre células do tabuleiro, o que exige examinar todas as direções possíveis para cada célula não bloqueada. A construção completa consome $\mathcal{O}(NM)$.
- Para cada um dos k peões a serem capturados, eu executo uma BFS com custo $\mathcal{O}(NM)$, o que totaliza $\mathcal{O}(k \cdot NM)$ no pior caso.

Portanto, a complexidade de tempo da solução aproximada é:

$$\mathcal{O}(k \cdot NM)$$

4.2 Espaço

4.2.1 Solução Exata

A solução exata armazena:

- $k+1$ matrizes de distância de tamanho NM, totalizando $\mathcal{O}((k+1) \cdot NM)$
- A matriz de distâncias reduzida de tamanho $(k+1) \times (k+1)$, que é desprezível
- A tabela de programação dinâmica de tamanho $2^k \cdot k$

Logo, a complexidade espacial é:

$$\mathcal{O}((k+1) \cdot NM + 2^k \cdot k)$$

4.2.2 Solução Aproximada

A solução aproximada armazena:

- Um grafo com até NM vértices e, no máximo, $8NM$ arestas (considerando 8 direções)
- Uma lista de adjacência com custo $\mathcal{O}(NM)$
- Um vetor de descobertas e layers para cada BFS, com custo $\mathcal{O}(NM)$

Portanto, o uso de memória da solução aproximada é:

$$\mathcal{O}(NM)$$

5 Discussão dos Algoritmos

Nesta seção, eu comparo os dois algoritmos implementados quanto à qualidade da solução obtida e ao tempo de execução. Ambos foram testados com as mesmas instâncias de entrada, variando o número de peões e a densidade de obstáculos no tabuleiro.

Qualidade da Solução

A solução exata, baseada no algoritmo de Held-Karp com programação dinâmica e bitmask, encontra a sequência ótima de movimentos da rainha para capturar todos os peões com o menor número possível de jogadas. Dessa forma, ela serve como referência para a avaliação da solução aproximada.

A solução aproximada, por sua vez, segue uma abordagem gulosa. Em cada passo, ela realiza uma busca em largura (BFS) para encontrar o peão mais próximo (no sentido de número mínimo de jogadas da rainha) e move-se diretamente até ele. Por capturar sempre o peão mais próximo, essa abordagem fornece um limite superior para o número mínimo de jogadas. A resposta retornada por esse algoritmo nunca será menor do que a ótima, mas em muitos casos se aproxima bastante dela.

Na prática, eu observei que o erro relativo entre a resposta do algoritmo guloso e a exata foi pequeno em diversas instâncias, especialmente quando os peões estavam bem distribuídos e não havia muitos obstáculos separando-os.

Tempo de Execução

A principal limitação da solução exata é sua complexidade exponencial no número de peões, com tempo de execução $\mathcal{O}(2^k \cdot k^2)$, o que a torna inviável para valores altos de k (como $k > 16$). Em contrapartida, a solução aproximada executa em tempo polinomial, com $\mathcal{O}(k \cdot NM)$, e se mostrou eficiente mesmo para instâncias grandes (como tabuleiros de 100×100 com 16 peões).

A seguir, apresento uma comparação resumida entre os algoritmos e uma com suas respostas e tempos de execução (nos 8 testes disponibilizados no Moodle):

Critério	**Solução Exata**	**Solução Aproximada**
Precisão	Ótima (mínimo garantido)	Boa, mas não garantida
Tempo de execução	Exponencial ($2^k \cdot k^2$)	Polinomial ($k \cdot NM$)
Uso de memória	Alto ($2^k \cdot k + k \cdot NM$)	Baixo (NM)
Escalabilidade	Limitada a $k \leq 16$	Escalável para grandes tabuleiros

Exata (Held-Karp)		**Aproximada (Guloso)**	
Tempo (s)	**Resposta**	**Tempo (s)**	**Resposta**
0.023	8	0.029	8
0.107	22	0.198	25
0.313	34	0.227	36
0.645	42	0.357	43
1.639	43	0.386	49
2.833	57	0.379	58
5.854	53	0.438	53
12.570	51	0.366	58

Considerações Finais

A coexistência das duas abordagens foi essencial para equilibrar precisão e desempenho. A solução exata é indispensável para validação e testes em instâncias pequenas, enquanto a solução aproximada se mostrou eficaz em situações práticas com restrições de tempo e memória.

A abordagem gulosa, além de eficiente, fornece uma estimativa segura do número mínimo de jogadas, reforçando sua utilidade como heurística rápida para problemas maiores.

6 Referências

- Almeida, J. (2025). Slides virtuais da disciplina de algoritmos 1. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.